

Section 1: The Must-Dos

Before you start:

Create a class called **Utilities** in a file called **Utilities.java**. This should be placed in a package called **utils**.

Standalone methods

Exercise 1) Calculating the absolute value of a number

Note: The absolute value of a number is the positive version of that number, e.g. the absolute value of -5 is 5

Write a **static method** called **getAbs()** in your **Utilities** class that calculates the absolute value of a supplied number. It should:

- 1) Take in one parameter, a **double** containing the number to be worked with
- 2) Calculate the absolute value of the supplied number
 - a. If the value is ≥ 0 , return that original value
 - b. If the value is < 0 , return the value $\times -1$
- 3) Return a **double**: the absolute value

Test your code by creating a new class (**UtilitiesTestBed** in **UtilitiesTestBed.java**) in a package called **apps**. Call your **getAbs()** method with a hard-coded value (e.g. -5) and display the result on the console.

Examples:

- **getAbs(-5)** should return 5
- **getAbs(5)** should return 5
- **getAbs(0)** should return 0

Commit and push your code when your method returns the correct value for each of the above scenarios

Exercise 2) Conversion from celsius to fahrenheit

Write a **static method** called **toFahrenheit()** in your **Utilities** class that converts a temperature from Celsius to Fahrenheit. It should:

- 4) Take in one parameter, a **double** containing the temperature in Celsius
- 5) Convert that temperature to Fahrenheit
 - a. Conversion formula: **celsiusTemp * (9/5) + 32**
- 6) Return a **double**: the converted value

Test your code in your **UtilitiesTestBed** by calling your **toFahrenheit()** method with a hard-coded value (e.g. 23, as in the example below) and display the result on the console.

Example: **toFahrenheit(23)** should return 73.4

Commit and push your code when your method returns a correct value

Building methods from other methods

Exercise 3) Validating a password

Part 1: Checking for an uppercase character

Write a **static method** called **containsUpperCase()** in your **Utilities** class that checks if a supplied String parameter contains at least one uppercase letter. It should:

- 1) Take in one parameter, a **String** containing a piece of text
- 2) Check that the piece of text contains an uppercase letter by:
 - a. Looping through each character in the String
 - i. Using **text.charAt(i)** in a for loop will loop through each character in the String
 - b. Checking if the current character is in uppercase
 - i. **Character.isUpperCase(currentChar)** can be used to do this
- 3) Return a **boolean**:
 - a. **True** if an uppercase character is found
 - b. **False** if no uppercase character is found

Test your code in your **UtilitiesTestBed** class by calling your **containsUpperCase()** method with the following hard-coded values and displaying each of the result on the console:

Examples:

- `containsUpperCase("hello")` should return false (no uppercase character present)
- `containsUpperCase("Hello")` should return true
- `containsUpperCase("")` should return false – it should not crash!

Commit and push your code when your method returns the correct result for each of the above examples.

Part 2: Validate a password

Write a **static method** called **validatePassword()** in your **Utilities** class that checks if a supplied String has 8 or more characters and contains at least one uppercase letter. It should:

- 1) Take in one parameter, a **String** containing a password
- 2) Check that the password meets the following criteria:
 - a. Is at least 8 characters long
 - b. Contains at least one uppercase character
 - i. Use your **containsUpperCase()** method from part 1 to check this
- 3) Return a **boolean**:
 - a. **True** if the supplied password is 8 characters or longer AND includes an uppercase letter
 - b. **False** if the supplied password is shorter than 8 characters or does not include an uppercase

Test your code in your **UtilitiesTestBed** class by calling your **validatePassword()** method with the following hard-coded values and displaying each of the result on the console:

Examples:

- `validatePassword("Hello")` should return false
- `validatePassword("Hello there")` should return false (long enough, but there's no uppercase or number)
- `validatePassword("Hello There")` should return true (long enough and has an uppercase)

Commit and push your code when your method returns the correct result for each of the above examples.

Exercise 4) Checking for vowels

Part 1: Counting vowels in a String

Write a **static method** called **countVowels()** in your **Utilities** class that counts the number of vowels in a supplied string. **Note: This method should not be case sensitive! It should:**

- 1) Take in one parameter, a **String** containing the text in which to count the vowels
- 2) Loop through the String and counts how many vowels appear
 - a. Note: For this method, only count the **classic** vowels: "a", "e", "i", "o" and "u"
- 3) Return an **int**: the number of vowels found in the supplied text

Test your code in your **UtilitiesTestBed** by calling your **countVowels()** method with the hard-coded values shown in the examples below and display the result on the console.

Examples:

- `countVowels("")` should return 0 (Empty string)
- `countVowels("Hmmm")` should return 0 (No vowels present)
- `countVowels("Hi there")` should return 3 (all lowercase)
- `countVowels("I like it!")` should return 4 (3 lowercase vowels and 1 uppercase)
- `countVowels("OOOOO")` should return 6 (6 uppercase)

Commit and push your code when your method returns a correct value for all the above examples

Part 2: Checking for all vowels

Write a **static method** called **isAllVowels()** in your **Utilities** class that checks if a supplied string contains **ONLY** vowels. **Note: This method should not be case sensitive! It should:**

- 4) Take in one parameter, a **String** containing the text to be checked
- 5) Check if the string contains only vowels
 - a. Note: Consider using your `countVowels()` method as part of this logic
- 6) Return a **boolean**:
 - a. True if the text only contains vowels
 - b. False if the text contains any non-vowels

Test your code in your *UtilitiesTestBed* by calling your `isAllVowels()` method with the hard-coded values shown in the examples below and display the result on the console.

Examples:

- `isAllVowels("")` should return false (Empty string)
- `isAllVowels("Hmmm")` should return false (No vowels present)
- `isAllVowels("Hi there")` should return false (contains a mix of vowels AND consonants)
- `isAllVowels("OOooooOO")` should return true (all characters are vowels, in a mix of upper and lower case)
- `isAllVowels("OOOOOO")` should return true (all characters are vowels in uppercase)

Commit and push your code when your method returns a correct value for all the above examples

Section 2: For extra practice

Before you start:

Create a class called **Basket** in a file called **Basket.java**. This should be placed in a package called **services**.

Create a class called **Shop** in a file called **Shop.java**. This should be placed in your **apps** package.

Shopping basket

A shop stores its shopping basket information in three **parallel** arraylists:

- 1) An `ArrayList<String>` `itemNamees` holding all product names
- 2) An `ArrayList<int>` `itemQuantities` holding all product quantities
- 3) An `ArrayList<double>` `itemPrices` holding all product prices

Create these three lists in your **Shop** class and fill them with some hard-coded values.

Part 1: Calculating the subtotal of an item in the basket

Write a **static method** called `calcSubtotal()` in your **Basket** class that calculates the subtotal for a specific item based on its quantity and price. It should:

- 1) Take in two parameters:
 - a. An **int** containing the **quantity** of each item in the basket
 - b. A **double** containing the **price** of each item in the basket
- 2) Return a **double**: the price * the quantity

Update your **Shop** class to display the subtotal for every item in your shopping basket on the console. You should use your `calcSubtotal()` method to do this.

Commit and push your code when your method returns correct values for each of your items in your basket

Part 2: Displaying an item from the basket (including its subtotal)

Write a **static void method** called `displayItemLine()` in your **Basket** class that outputs the details of an item to the console, including its subtotal cost. It should:

- 1) Take in three parameters:
 - a. A **String** containing the name of an item in the basket
 - b. A **int** containing the **quantity** of an item in the basket
 - c. A **double** containing the **price** of an item in the basket
- 2) Display a nicely formatted piece of text indicating:
 - a. The name of the item
 - b. How many units are being purchased
 - c. How much each unit costs
 - d. The subtotal for this item (you should use your `calcSubtotal()` method to calculate this)
- 3) Return nothing (i.e. it is a **void method**)

Update your **Shop** class to use your `displayItemLine()` method to display the subtotal for every item in your shopping basket on the console.

Commit and push your code when your method displays the correct details for each item in your basket

Part 3: Displaying all items in the basket (including the basket's total)

Write a **static void method** called `displayBasket()` in your **Basket** class that outputs the details of every item in the basket to the console, then displays the final total. It should:

- 1) Take in three parameters:

- a. An **ArrayList<String>** containing the names of the items in the basket
 - b. An **ArrayList<int>** containing the quantities of the items in the basket
 - c. An **ArrayList<double>** containing the prices of the items in the basket
- 2) Display each of the items in the basket (you should use your **displayItemLine()** method to do this)
 - 3) Display the total price for the basket. Note: you can do this in two different ways:
 - a. Maintain a running total in this method – add the subtotal for each item to this
 - b. Write a separate method to calculate the cost of the whole basket.
 - 4) Return nothing (i.e. it is a **void method**)

Update your Shop class to use your **displayBasket()** method to display the details of your shopping basket on the console.

Commit and push your code when your method displays the correct details for everything in your basket